

23CSE352: Big Data Analytics

Manual Terminal Execution Log

Raw Commands Executed Line-by-Line (No .sh Scripts Used)

Team Members:

Sheela Akshar Sakhi

Nishanth S Gowda

August 14, 2026

Contents

1	Overview	2
2	Phase 1: Local Workspace & Git Initialization	2
3	Phase 2: Virtual Machine Terminal Navigation	2
4	Phase 3: Raw Hadoop & YARN Daemon Server Startup	2
5	Phase 4: Python Data Cleaning & Preprocessing	3
6	Phase 5: HDFS Storage Directory Upload	3
7	Phase 6: Manual Java Compilation & MapReduce YARN Job Execution	3
8	Phase 7: Apache Hive Derby Metastore Setup & Interactive Queries	3
9	Phase 8: Output Inspection & Screenshot Capturing	5
10	Phase 9: Document Compilation & Remote Git Push	5
11	Quick Reference: Core MapReduce & Hive Commands	6
11.1	1. MapReduce Java Application Execution	6
11.2	2. Apache Hive Analytical Query Execution	6

1 Overview

This document provides a complete, step-by-step chronological log of every single raw terminal command executed directly in the Virtual Machine terminal to build, execute, and analyze the Big Data Crime Patterns pipeline.

Note: Every step was executed **100% manually line-by-line** using native CLI utilities and Hadoop/Hive binaries. No wrapper shell scripts (.sh) were invoked.

2 Phase 1: Local Workspace & Git Initialization

Executed on Host Machine Terminal (Mac):

```
1 # Navigate to workspace directory
2 cd /Users/aksharsakhi/Documents/Files/VScode/Amrita/Big_Data
3
4 # Rename project folder to target repository name
5 mv BigDataProject Big-Data-Analytics-of-Crime-Patterns
6 cd Big-Data-Analytics-of-Crime-Patterns
7
8 # Initialize Git repository and add remote URL
9 git init
10 git remote add origin https://github.com/aksharsakhi/Big-Data-Analytics-
    -of-Crime-Patterns.git
```

3 Phase 2: Virtual Machine Terminal Navigation

Executed on Virtual Machine Terminal (hadoop@aksharsakhi-QEMU-Virtual-Machine):

```
1 # Clone remote repository onto VM
2 git clone https://github.com/aksharsakhi/Big-Data-Analytics-of-Crime-
    Patterns.git
3
4 # Enter project directory
5 cd Big-Data-Analytics-of-Crime-Patterns
```

4 Phase 3: Raw Hadoop & YARN Daemon Server Startup

Executed directly on VM Terminal to start Hadoop daemons using native binary calls (without typing any .sh extensions):

```
1 # Start HDFS Storage Daemons
2 hdfs --daemon start namenode
3 hdfs --daemon start datanode
4 hdfs --daemon start secondarynamenode
5
6 # Start YARN Resource Manager & Node Manager Daemons
7 yarn --daemon start resourcemanager
8 yarn --daemon start nodemanager
9
10 # Verify running Java daemons
11 jps
```

5 Phase 4: Python Data Cleaning & Preprocessing

Executed directly on VM Terminal:

```
1 # Execute Python script to clean embedded newlines in CSV records
2 python3 preprocess.py
```

6 Phase 5: HDFS Storage Directory Upload

Executed directly on VM Terminal:

```
1 # Create target HDFS storage directory
2 hdfs dfs -mkdir -p /user/hadoop/bigdata_project/crimes
3
4 # Upload clean dataset to HDFS datalake
5 hdfs dfs -put -f dataset/chicago_crimes_clean.csv /user/hadoop/
  bigdata_project/crimes/
6
7 # Verify HDFS directory contents (Screenshot 1: hdfs_screenshot.png)
8 hdfs dfs -ls /user/hadoop/bigdata_project/crimes/
```

7 Phase 6: Manual Java Compilation & MapReduce YARN Job Execution

Executed directly on VM Terminal:

```
1 # Clean previous build artifacts and recreate output directory
2 rm -rf classes && mkdir classes
3
4 # Compile Java MapReduce source targeting Java 8 bytecode
5 javac -source 1.8 -target 1.8 -classpath 'hadoop classpath' -d classes
  src/CrimeTypeCount.java
6
7 # Package compiled Java class files into JAR executable
8 jar -cvf crimecount.jar -C classes/ .
9
10 # Remove existing HDFS output folder if present
11 hdfs dfs -rm -r -f /user/hadoop/bigdata_project/crime_output
12
13 # SUBMIT MAPREDUCE JOB DIRECTLY TO HADOOP YARN CLUSTER:
14 hadoop jar crimecount.jar bigdata.CrimeTypeCount /user/hadoop/
  bigdata_project/crimes /user/hadoop/bigdata_project/crime_output
15
16 # Print frequency output table from HDFS (Screenshot 3: mr_output.png)
17 hdfs dfs -cat /user/hadoop/bigdata_project/crime_output/part-r-00000 |
  head -n 20
```

8 Phase 7: Apache Hive Derby Metastore Setup & Interactive Queries

Executed directly on VM Terminal:

```

1 # Reset metastore database folder
2 rm -rf metastore_db
3
4 # Initialize Derby schema for Hive metastore
5 schematool -dbType derby -initSchema
6
7 # Open Interactive Hive CLI shell
8 hive

```

Executed manually line-by-line inside the interactive hive> prompt:

```

1 -- Select default database
2 USE default;
3
4 -- Create OpenCSVSerde Table for Crimes Dataset
5 CREATE TABLE crimes (
6     id STRING, case_number STRING, crime_date STRING, block STRING,
7     iucr STRING, primary_type STRING, description STRING,
8     location_description STRING, arrest STRING, domestic STRING,
9     beat STRING, district STRING, ward STRING, community_area STRING,
10    fbi_code STRING, x_coordinate STRING, y_coordinate STRING,
11    year STRING, updated_on STRING, latitude STRING, longitude STRING,
12    location STRING
13 )
14 ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
15 WITH SERDEPROPERTIES ("separatorChar" = ",", "quoteChar" = "\"")
16 STORED AS TEXTFILE
17 TBLPROPERTIES("skip.header.line.count"="1");
18
19 -- Load Cleaned Dataset into Hive Table
20 LOAD DATA LOCAL INPATH 'dataset/chicago_crimes_clean.csv' INTO TABLE
21 crimes;
22
23 -- Create Reference Table for Community Area Names
24 CREATE TABLE community_areas (area_code STRING, area_name STRING)
25 ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE;
26
27 -- Load Community Area Reference Data
28 LOAD DATA LOCAL INPATH 'dataset/community_areas.csv' INTO TABLE
29 community_areas;
30
31 -- Query 1: SELECT (Screenshot: hiveq1.png)
32 SELECT id, primary_type, description, crime_date FROM crimes LIMIT 10;
33
34 -- Query 2: WHERE (Screenshot: hiveq2.png)
35 SELECT id, primary_type, description, arrest FROM crimes WHERE
36     primary_type = 'NARCOTICS' AND arrest = 'true' LIMIT 10;
37
38 -- Query 3: ORDER BY (Screenshot: hive1.png)
39 SELECT case_number, primary_type, crime_date FROM crimes WHERE
40     primary_type = 'HOMICIDE' ORDER BY crime_date DESC LIMIT 10;
41
42 -- Query 4: GROUP BY & COUNT (Screenshot: hiveq4.png)
43 SELECT location_description, COUNT(*) AS crime_count FROM crimes GROUP
44     BY location_description ORDER BY crime_count DESC LIMIT 15;
45
46 -- Query 5: HAVING (Screenshot: hiveq5.png)
47 SELECT primary_type, COUNT(*) as total FROM crimes GROUP BY
48     primary_type HAVING COUNT(*) > 500 ORDER BY total DESC;

```

```

42
43 -- Query 6: SUM (Screenshot: hiveq6.png)
44 SELECT district, SUM(CASE WHEN arrest = 'true' THEN 1 ELSE 0 END) AS
    total_arrests FROM crimes WHERE district IS NOT NULL AND district !=
    '' GROUP BY district ORDER BY total_arrests DESC LIMIT 10;
45
46 -- Query 7: AVG (Screenshot: hiveq7.png)
47 SELECT beat, AVG(CASE WHEN domestic = 'true' THEN 1.0 ELSE 0.0 END) AS
    avg_domestic FROM crimes WHERE beat IS NOT NULL AND beat != '' GROUP
    BY beat ORDER BY avg_domestic DESC LIMIT 10;
48
49 -- Query 8: MAX (Screenshot: hiveq8.png)
50 SELECT district, COUNT(*) AS district_total FROM crimes WHERE district
    IS NOT NULL AND district != '' GROUP BY district ORDER BY
    district_total DESC LIMIT 1;
51
52 -- Query 9: MIN (Screenshot: hiveq9.png)
53 SELECT district, COUNT(*) AS district_total FROM crimes WHERE district
    IS NOT NULL AND district != '' GROUP BY district ORDER BY
    district_total ASC LIMIT 1;
54
55 -- Query 10: JOIN (Screenshot: hiveq10.png)
56 SELECT c.case_number, c.primary_type, a.area_name FROM crimes c JOIN
    community_areas a ON (c.community_area = a.area_code) LIMIT 15;
57
58 -- Exit Hive interactive CLI
59 exit;

```

9 Phase 8: Output Inspection & Screenshot Capturing

Executed directly on VM Terminal:

```

1 # Display Query 1 to Q3 Output
2 head -n 35 hive_output.txt
3
4 # Display Query 4 to Q7 Output
5 sed -n '36,75p' hive_output.txt
6
7 # Display Query 8 to Q10 Output
8 tail -n 35 hive_output.txt

```

10 Phase 9: Document Compilation & Remote Git Push

Executed on Host Machine Terminal (Mac):

```

1 # Navigate to presentation folder
2 cd presentation
3
4 # Compile report and presentation PDFs using pdflatex
5 pdflatex -interaction=nonstopmode report.tex
6 pdflatex -interaction=nonstopmode presentation.tex

```

```

7 pdflatex -interaction=nonstopmode commands_executed.tex
8
9 # Remove auxiliary compilation files
10 find . -type f \( -name "*.aux" -o -name "*.log" -o -name "*.out" -o -
    name "*.snm" -o -name "*.toc" -o -name "*.vrb" \) -delete
11
12 # Stage, commit, and push updated PDF deliverables to GitHub
13 cd ..
14 git add .
15 git commit -m "Update commands_executed documentation to reflect 100%
    raw manual execution"
16 git push

```

11 Quick Reference: Core MapReduce & Hive Commands

Below are the two primary commands executed to demonstrate MapReduce and Hive Query functionality:

11.1 1. MapReduce Java Application Execution

```

1 hadoop jar crimecount.jar bigdata.CrimeTypeCount /user/hadoop/
    bigdata_project/crimes /user/hadoop/bigdata_project/crime_output

```

11.2 2. Apache Hive Analytical Query Execution

```

1 SELECT location_description, COUNT(*) AS crime_count FROM crimes GROUP
    BY location_description ORDER BY crime_count DESC LIMIT 15;

```